Lab 2: [numerical and process-based modelling]

Kohki Kawata (579554839)

2026-08-10

Setup

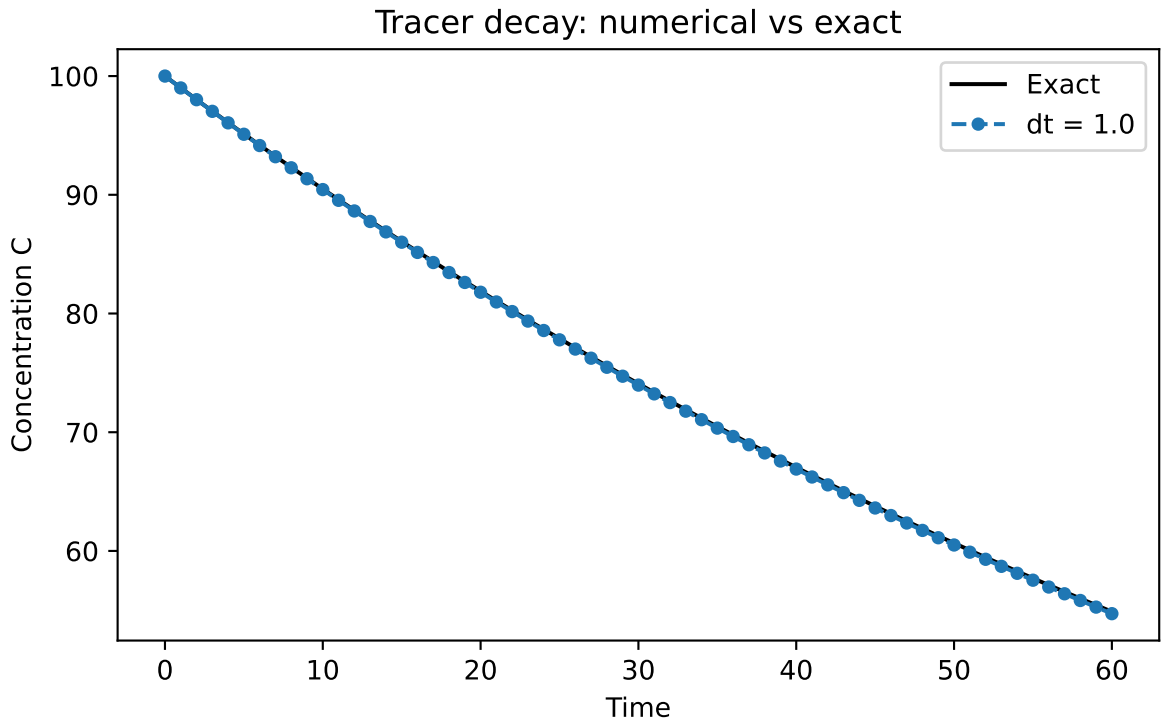
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Part A: [tracer decay and step size]

Write one sentence describing what this part is doing.

```
def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
    ts = [0.0]
    Cs = [C0]
    for i in range(n):
        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
        Cs.append(C)
    return np.array(ts), np.array(Cs)

# The exact answer, for comparison
C0, k, t_end = 100.0, 0.01, 60
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

fig, ax = plt.subplots(figsize=(7, 4))
```

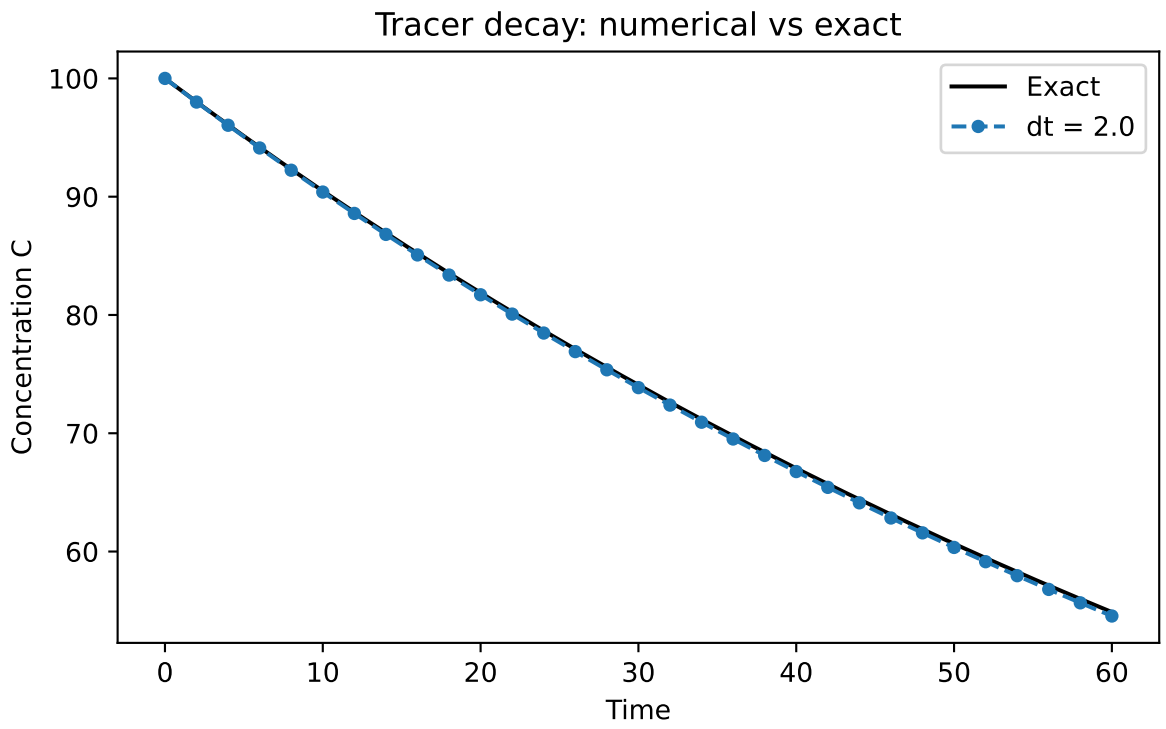
```
ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [1.0]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()
# The e
```



Answer to task A1: *(one to three sentences)* k rate controls the gradient of the curve.

```
def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
    ts = [0.0]
```

```
    Cs = [C0]
    for i in range(n):
        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
        Cs.append(C)
    return np.array(ts), np.array(Cs)

# The exact answer, for comparison
C0, k, t_end = 100.0, 0.01, 60
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [2.0]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()
# The e
```

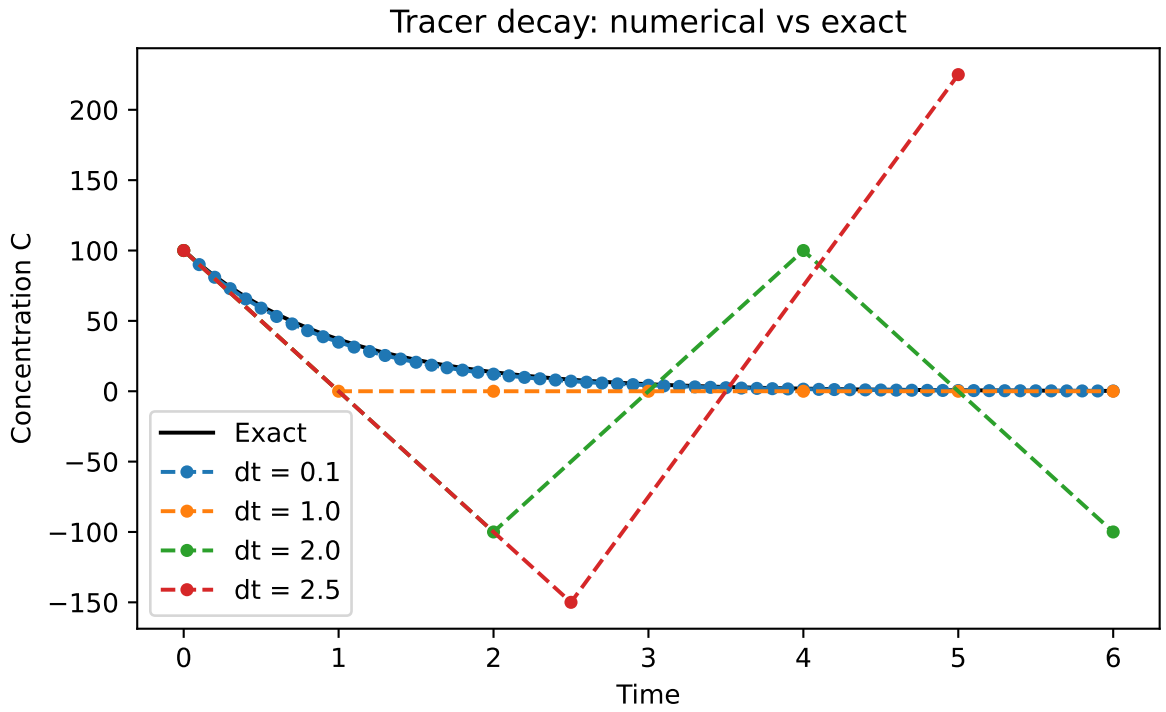


Answer to task A2: *(one to three sentences)*

the disagreement happens at dt value of 2.0

```
def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
    ts = [0.0]
    Cs = [C0]
    for i in range(n):
        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
        Cs.append(C)
    return np.array(ts), np.array(Cs)

# The exact answer, for comparison
C0, k, t_end = 100.0, 1.0, 6.0
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)
```

```
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [0.1, 1.0, 2.0, 2.5]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()
# The e
```

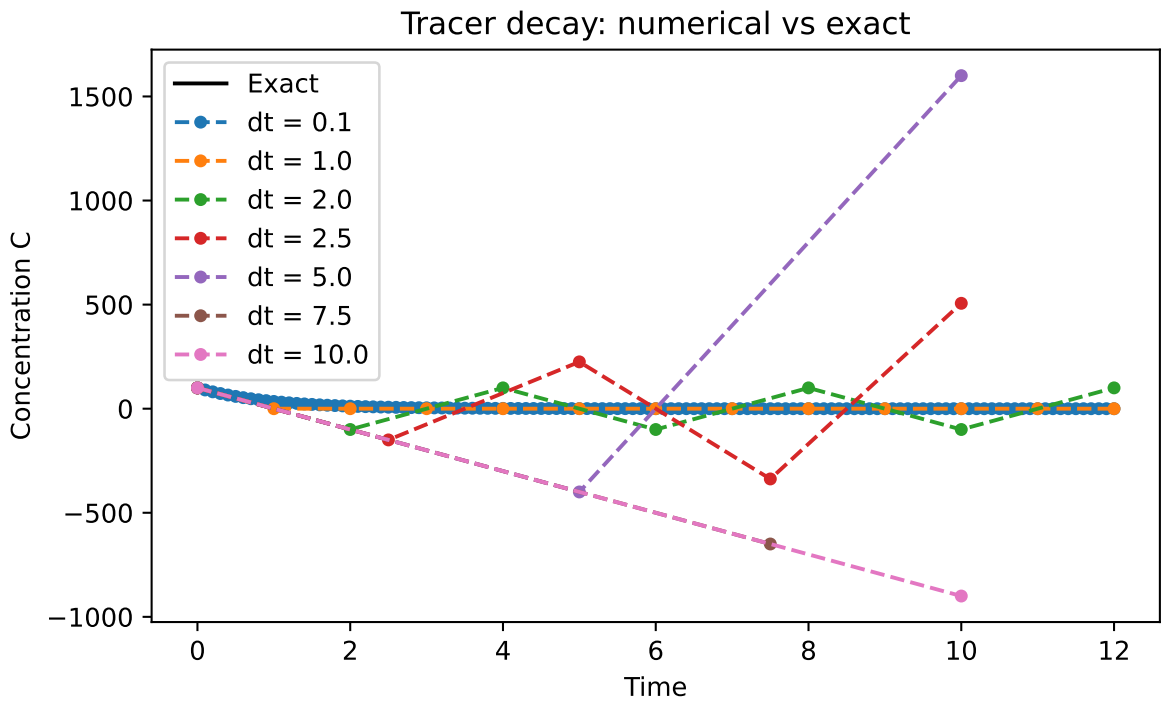


Answer to task A3: *(one to three sentences)*

the largest dt that still gives a non-negative is 1.0

```
def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
```

```
    C = C0
    ts = [0.0]
    Cs = [C0]
    for i in range(n):
        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
        Cs.append(C)
    return np.array(ts), np.array(Cs)

# The exact answer, for comparison
C0, k, t_end = 100.0, 1.0, 12.0
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [0.1, 1.0, 2.0, 2.5, 5.0 , 7.5, 10.0]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()
# The e
```

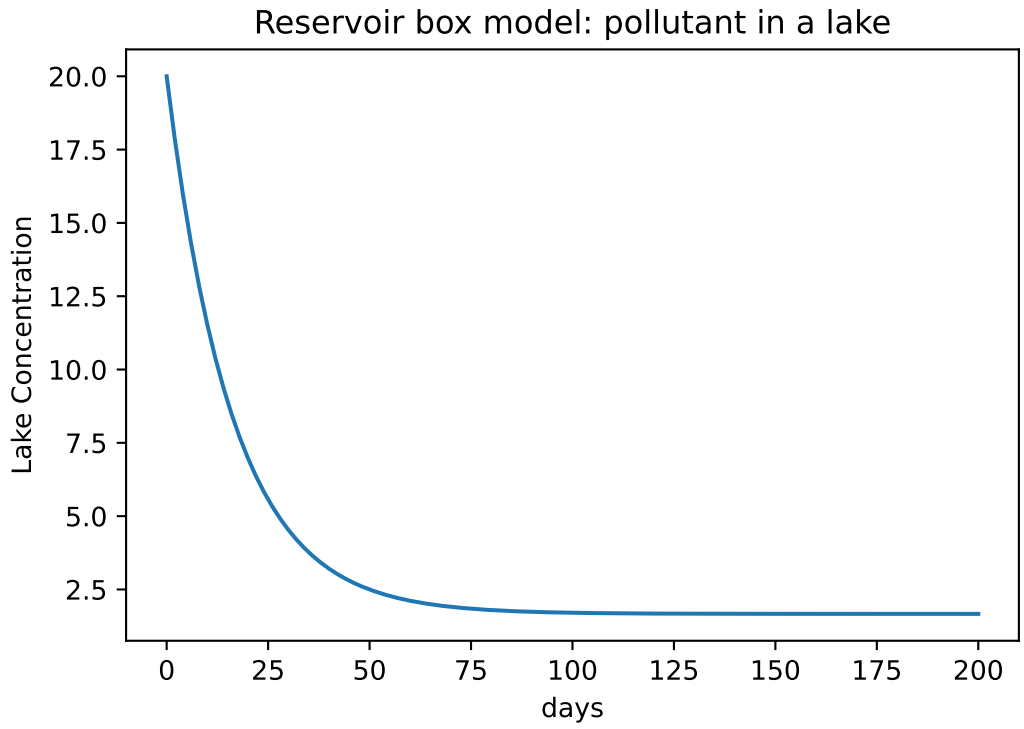


Answer to task A4: *(one to three sentences)* When dt is large, the numerical solution becomes a negative due to the equation multiplying the value with the C old value, the larger the dt number the larger the negative. This is then over compensated and shoots positive. the larger the dt value, the larger the oscilation. ## Part B: [reservoir box model]

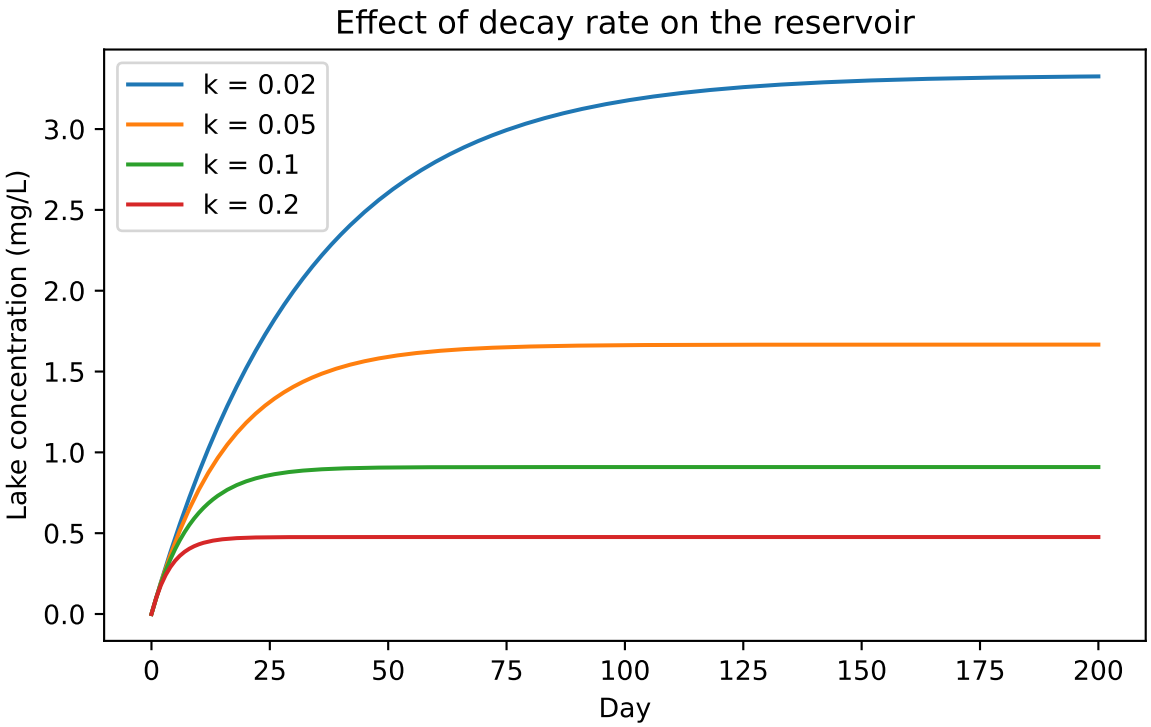
```
import numpy as np
import matplotlib.pyplot as plt

# ---- Parameters ----
V = 1.0e6        # lake volume (m^3)
Q = 1.0e4        # river inflow (m^3 per day)
C_in = 10.0      # incoming concentration (mg/L)
k = 0.05         # in-lake decay rate (per day)

# ---- Time settings ----
dt = 1.0         # time step (days)
days = 200

# ---- Initial condition ----
C = 20.0          # clean lake

# ---- Storage ----
```

```
history = [C]

# ---- Time loop ----
for _ in range(days):
    dCdt = (Q * C_in - Q * C - k * V * C) / V   # the rate of change
    C = C + dCdt * dt                           # forward difference
    history.append(C)

# ---- A quick look at the numbers ----
print(f"Started at {history[0]:.2f} mg/L")
print(f"After 10 days: {history[10]:.2f} mg/L")
print(f"After 50 days: {history[50]:.2f} mg/L")
print(f"After {days} days: {history[-1]:.2f} mg/L")

fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(range(days + 1), history)
ax.set_xlabel("days")
ax.set_ylabel("Lake Concentration")
ax.set_title("Reservoir box model: pollutant in a lake")
plt.show()
C_star = (Q * C_in) / (Q + k * V)
print(f"Analytical steady state: {C_star:.2f} mg/L")
print(f"Numerical final value:   {history[-1]:.2f} mg/L")
print(f"Difference:              {abs(C_star - history[-1]):.4f} mg/L")

fig, ax = plt.subplots(figsize=(7, 4))

for k_test in [0.02, 0.05, 0.10, 0.20]:
    C = 0.0
    history = [C]
    for _ in range(days):
        dCdt = (Q * C_in - Q * C - k_test * V * C) / V
        C = C + dCdt * dt
        history.append(C)
    ax.plot(history, label=f"k = {k_test}")

ax.set_xlabel("Day")
ax.set_ylabel("Lake concentration (mg/L)")
ax.set_title("Effect of decay rate on the reservoir")
ax.legend()
plt.show()
```

```
Started at 20.00 mg/L
After 10 days: 11.54 mg/L
After 50 days: 2.50 mg/L
After 200 days: 1.67 mg/L
```




```
Analytical steady state: 1.67 mg/L
Numerical final value:   1.67 mg/L
Difference:              0.0001 mg/L
```



Answer to task B1: *(short answer)* Analytical steady state: 0.91 mg/L Numerical final value: 0.91 mg/L **Answer to task B2:** *(short answer)* concentration decreases over rapidly initially and gradually levels off reaching a steady state value of 0.91 mg/L. **Answer to task B3:** *(short answer)* steady rate increases from a 0.91 to a 1.67 mg/L. From the equation we can see that steady state is proportional to the inflow concentration, so if inflow concentration increases due to in flow increase, the steady state will also increase. **Answer to task B4:** *(short answer)* removal rate k affecs the denominator, hence a larger k value would decreases steady state concentration, similarly a higher k value would increase the rate of decay, allowing the concentration to reach steady state faster, showing a steeper speed of approach to steady state. ## Reflection The trickiest part of this lab was understanding the relationship between the decay rate and steady state concentration. But more importantly, the coding part of the assignment was challenging, especially the implementation of the numerical solution into understanding what each values did within the equation. Projecting my understanding through python was a challenge. If I were to improve one thing about my oart B codes, I would have allowed more elaborate answers to the questions, and also changed the code to allow for more flexibility in changing parameters.

Checklist before you push

- ☐ Every task has a code cell **and** a short prose answer

☐ Every figure has axis labels and a title
☐ Your name and student ID are in the YAML header
☐ The file renders cleanly (no red error boxes in the HTML output)
☐ Filename is `lab-XX-topic.qmd`, not `Untitled.qmd`